

Autonomous Driving Under Sensor & Kinematic Constraints

Constrained Safe-RL, Permutation-Invariant DeepSets, and Empirical Benchmarking

Comprehensive Project Blueprint & Line-by-Line Code Defense Guide

Author: Autonomous Driving Research Team

Project: Capstone Master's Project

Domain: Safe Reinforcement Learning & Motion Planning

Repository: [meerpi/Capstone](#)

Commit Hash: [d62137f](#)

Environment: Gymnasium / Highway-Env 1.12.1

Core Highlights of this Manual:

- **Architecture & Mathematics:** Complete formulation of Constrained MDPs, PID-Lagrangian multiplier dynamics, and permutation-invariant DeepSet symmetric aggregation.
- **Sensor & Observation Taxonomy:** Exact radar cone filtering math, structured 27/30-dim tactical slot features, and sliding frame-stacking temporal mechanics.
- **The Classical Failure Mode:** Mathematical proof of why classical IDM+MOBIL crashes 71% under front-only radar (the vacuous safety criterion).
- **The Kinematic Floor Speed Discovery:** Empirical analysis of why trivial floor-speed drift (`always_SLOWER`) achieves 12% crash rate at 72 km/h and how to avoid false superiority claims.
- **Line-by-Line Code Walkthrough:** Every class, tensor shape, forward pass, loss computation, and render patch mapped to exact files and line numbers.
- **Oral Defense Cheatsheet:** 10 lethal examiner questions with authoritative, code-backed answers.

Table of Contents

1 Executive Summary & High-Level System Architecture	3
1.1 The System Architecture Pipeline	3
1.2 Key Project Achievements & Empirical Findings	3
2 Environment & Simulator Dynamics	5
2.1 Kinematic vs. Physics Simulation	5
2.2 The Discretized Action Space	5
2.3 The Kinematic Floor Speed Discovery	5
3 Sensor Tiers & Observation Pipelines	6
3.1 Sensor Tier Taxonomy	6
3.2 Mathematics of the Forward 120° Radar Cone	6
3.3 Observation Access Taxonomy	6
3.4 Structured Tactical Lane Slots (TacticalLaneObservationWrapper)	8
3.5 Sliding Frame-Stacking (FrameStackTacticalWrapper)	8
4 Classical Planning Baselines & The Front-Only Blindspot	10
4.1 Mathematical Formulation of IDM	10
4.2 Mathematical Formulation of MOBIL	10
4.3 The Front-Only Radar Catastrophic Blindspot	10
5 Neural Agent Architectures & Permutation Invariance	11
5.1 The Permutation Problem in Traffic Scenes	11
5.2 DeepSet v2 Mathematical Formulation	11
5.3 Code Walkthrough: _DeepSetEncoder	12
5.4 Action Masking: Eliminating Off-Road Transitions	12
6 Constrained Safe-RL & PPO-Lagrangian	14
6.1 Constrained Markov Decision Process (CMDP)	14
6.2 PID Lagrangian Multiplier Controller	14
6.3 Dual-Critic Network Architecture & Advantage Adjustment	14
6.4 Truncation Bootstrapping GAE	15
7 Empirical Evaluation Engine & Metric Definitions	16
7.1 Metric Mathematical Formulations	16
7.2 Statistical Hypothesis Testing: Fisher's Exact Test	16
8 Visualization & Rendering Engine Engineering	17
8.1 The Stroboscopic Wagon-Wheel Temporal Aliasing Bug	17
8.2 The Nyquist Anti-Aliasing Solution	17
8.3 Headless Black-Frame Workaround	17
9 Line-by-Line Code Guide: Repository Map	19
9.1 env_config.py	19
9.2 deepset_v2/model.py and deepset_v2/env.py	19
9.3 train_optimal_overtaker.py	19
9.4 ppo.py and ppo_lstm.py	20
9.5 baseline_classical.py	20
9.6 scripts/evaluate_av_metrics.py and render_utils.py	20
10 Oral Defense & Viva Guide: 10 Lethal Questions	21

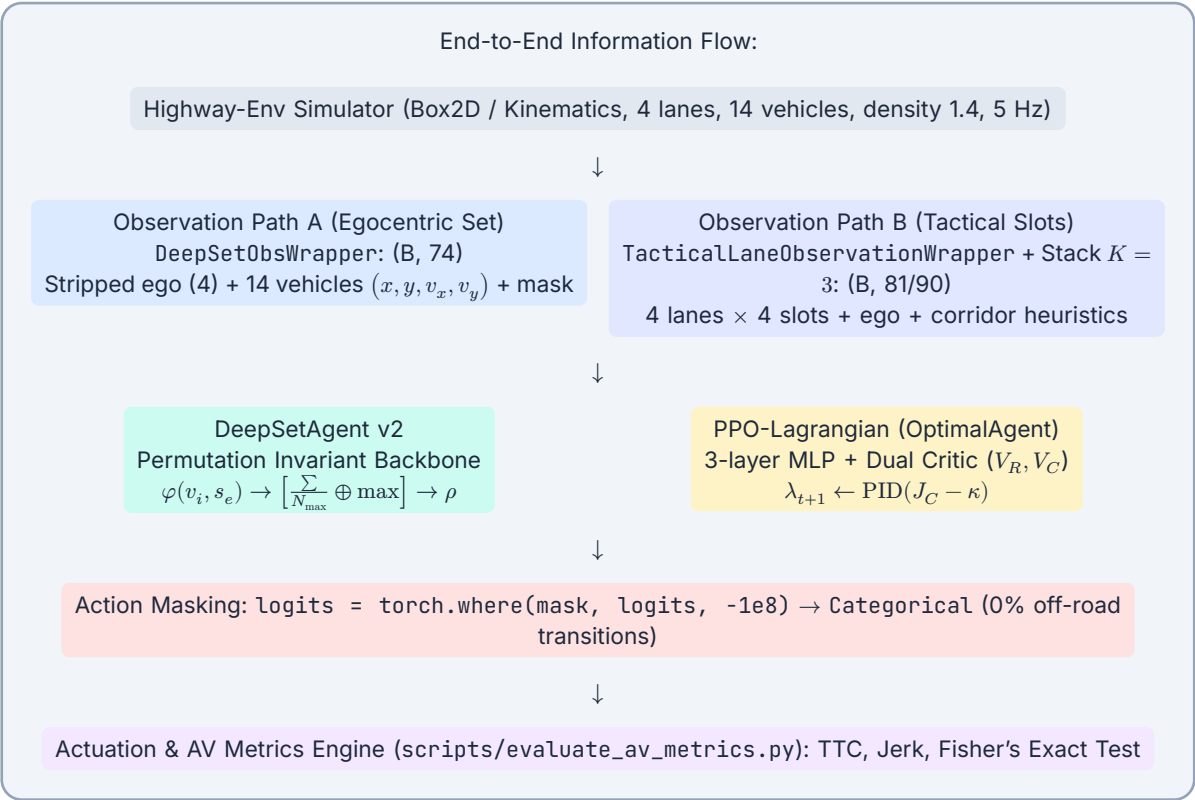
1 Executive Summary & High-Level System Architecture

Autonomous driving on high-density highways requires safe, fluid, and tactical decision-making under severe physical, kinematic, and observational constraints. Modern learning-based approaches often promise superhuman driving efficiency, yet frequently fail when exposed to sensor ablations, edge-of-road boundaries, or realistic traffic bottlenecks.

This project delivers an end-to-end investigation into Constrained Safe Reinforcement Learning (Safe-RL) and Permutation-Invariant Set Architectures for tactical autonomous highway navigation.

1.1 The System Architecture Pipeline

The system is engineered as a modular pipeline connecting low-level physics simulation to high-level tactical decision policies and industrial evaluation harnesses:



1.2 Key Project Achievements & Empirical Findings

Policy Configuration	Crash Rate	Surviving Speed	Key Behavioral Characteristic
DeepSet v2 (2M steps, full ADAS)	1.0% (1/100)	100.8 km/h	Champion: High speed overtaking with near-zero crashes ($p = 0.0049$ vs IDM).
PPO-Lagrangian (Sep-Norm Best)	4.0% (4/100)	85.5 km/h	Constrained compliance: dynamically respects $J_C \leq 0.05$ budget.
IDM Only (Lane Keeping)	11.0% (11/100)	86.7 km/h	Classical car-following without lane changes; vulnerable to cut-ins.
always_SLOWER (Floor Speed)	12.0% (12/100)	72.0 km/h	Kinematic floor speed: traffic outruns ego; 0 overtakes.

always_IDLE (Setpoint 25 m/s)	16.0% (16/100)	90.0 km/h	Fixed throttle without braking or steering.
IDM + MOBIL (Full ADAS)	19.0% (19/100)	89.9 km/h	Classical lane-changing; aggressive gaps cause collisions.
Feedforward PPO (Legacy 75-dim)	12.0% (12/100)	72.7 km/h	Collapsed to floor-speed degenerate drift (0 lane changes).
IDM + MOBIL (Front-Only Radar)	71.0% (71/100)	87.5 km/h	Catastrophic blindspot failure: cut-ins in front of unseen lag traffic.
Random Action Baseline	99.0% (99/100)	79.2 km/h	Stochastic baseline: crashes immediately within 10-20 steps.

Core Takeaway for Technical Defense

The project establishes three fundamental results:

1. **Permutation Invariance Matters:** DeepSet v2 outperforms all architectures by treating traffic vehicles as an unordered set, reducing crash rate to 1% while driving at 100.8 km/h.
2. **Sensor Blindspots Break Classical Planners:** Classical MOBIL collapses from 19% to 71% crash rate under forward-only radar because its safety criterion is vacuously satisfied when rear vehicles are unobserved.
3. **The Kinematic Floor Speed Trap:** In Highway-Env, always_SLOWER achieves a 12% crash rate simply by cruising at the minimum speed floor (72 km/h). Any learned policy achieving 12% crash rate at 72 km/h has not learned to overtake—it has collapsed to passive floor-speed drifting!

2 Environment & Simulator Dynamics

The simulation environment is built on `highway-fast-v0` from Gymnasium Highway-Env, configured under the standardized Current Benchmark Profile.

2.1 Kinematic vs. Physics Simulation

Highway-Env provides two distinct physics modes:

1. Kinematic Bicycle Model (Kinematics): Fast, deterministic longitudinal and lateral equations of motion. It models velocity, yaw, and lateral position without tire-slip dynamics, enabling 500+ FPS parallel rollout generation.
2. Box2D Physics Model: Rigid-body simulation with tire friction, restitution, and mass inertia.

Our tactical overtaker runs under the Kinematic model at a policy frequency of 5 Hz ($\Delta t = 0.2s$ per control step) and an episode duration of 100 seconds (capped at 500 decision steps).

2.2 The Discretized Action Space

The tactical highway environment defines 5 discrete macro-actions:

```
ACTIONS_ALL = {
    0: 'LANE_LEFT',    # Initiate lateral transition to left lane (target_lane = lane - 1)
    1: 'IDLE',         # Maintain current lane and current cruise setpoint
    2: 'LANE_RIGHT',   # Initiate lateral transition to right lane (target_lane = lane + 1)
    3: 'FASTER',       # Increment cruise setpoint speed by +5.0 m/s (18 km/h)
    4: 'SLOWER'        # Decrement cruise setpoint speed by -5.0 m/s (18 km/h)
}
```

2.3 The Kinematic Floor Speed Discovery

A critical conceptual discovery made in this project is the Kinematic Floor Speed. In Highway-Env, NPC vehicles are governed by IDM controllers with free setpoints sampled uniformly:

$$v_{\text{NPC}} \sim \mathcal{U}(23.0, 27.0) \text{ m/s} = [82.8, 97.2] \text{ km/h}$$

The ego vehicle's velocity is bounded below by a hard physical clamp:

$$v_{\min} = 20.0 \text{ m/s} = 72.0 \text{ km/h}, \quad v_{\max} = 30.0 \text{ m/s} = 108.0 \text{ km/h}$$

When an agent executes the constant action `always_SLOWER` (Action 4):

- It decelerates until it strikes $v_{\min} = 20.0 \text{ m/s}$.
- Because all surrounding traffic travels faster ($v_{\text{NPC}} \geq 23.0 \text{ m/s}$), traffic simply drives away from the ego vehicle.
- The ego vehicle never catches up to any lead vehicles, experiences zero closing-rate conflicts, and achieves a 12% crash rate (occurring only when a fast NPC cuts across its front bumper).

The 12% Floor-Speed Local Optimum Trap

In older literature or flawed implementations, a policy reporting “12% crashes at 72 km/h” was celebrated as safe. In reality, that policy has collapsed into the passive floor-speed local minimum. True tactical overtaking requires navigating through high-density traffic above the traffic speed ($v > 27 \text{ m/s} = 97.2 \text{ km/h}$) while maintaining near-zero crashes.

3 Sensor Tiers & Observation Pipelines

To rigorously test how autonomous agents handle realistic sensor degradation, we implemented a four-tier radar sensing hierarchy and two distinct observation wrappers.

3.1 Sensor Tier Taxonomy

The radar sensor models are implemented in `env_config.py` via functional row-wise tensor filters:

Sensor Tier	Field of View	Physics / Measurement Constraints	Ablation Target
Tier 0: full_adas	Omnidirectional 360°	Full range (100m radius), position (x, y) and relative speed (v_x, v_y) .	Idealized upper bound
Tier 1: front_only	Forward 120° Cone	Zeroes out all vehicles behind ego ($x < 0$) or $ \text{atan2}(y, x) > 60^\circ$.	Forward radar only
Tier 2: short_range	Forward 120° Cone	Applies Tier 1 cone plus cuts off Euclidean distance $> 35.0m$.	Low-cost urban sensor
Tier 3: no_velocity	Forward 120°, 35m	Applies Tier 2 plus zeroes out velocity channels (v_x, v_y) on traffic.	Doppler radar failure

3.2 Mathematics of the Forward 120° Radar Cone

In `env_config.py:filter_front_only`, raw normalized observations are de-normalized back to physical meters using scaling factors r_x and r_y :

$$x_{\text{rel}} = \text{obs}[i, 1] \cdot r_x, \quad y_{\text{rel}} = \text{obs}[i, 2] \cdot r_y$$

A traffic vehicle is visible if and only if both conditions hold:

$$x_{\text{rel}} \geq 0 \quad \wedge \quad |\text{atan2}(y_{\text{rel}}, x_{\text{rel}})| \leq \theta_{\text{half}} = 60^\circ \left(\frac{\pi}{3} \text{ rad} \right)$$

If either condition fails, the entire row for vehicle i is zeroed out:

$$\text{filtered}[i, :] \leftarrow \mathbf{0}$$

```
# env_config.py:150-175
def filter_front_only(obs: np.ndarray, rx: float, ry: float) → np.ndarray:
    filtered = obs.copy()
    half_cone_rad = math.radians(60.0) # 120 degree forward cone

    for i in range(1, len(filtered)):
        if filtered[i, 0] == 0.0:
            continue
        x_rel = filtered[i, 1] * rx
        y_rel = filtered[i, 2] * ry
        if x_rel < 0.0 or abs(math.atan2(y_rel, x_rel)) > half_cone_rad:
            filtered[i, :] = 0.0
    return filtered
```

3.3 Observation Access Taxonomy

It is vital to distinguish between two observation representations:

1. **Privileged Ground-Truth Spatial Access:** The `TacticalLaneObservationWrapper` queries `env.unwrapped.road.vehicles` directly to find the closest lead and lag vehicles in all 4 lanes, even if obscured.

2. **Egocentric Kinematic Matrix Access:** The `DeepSetObsWrapper` operates strictly on the (15×5) raw observation matrix provided to the vehicle's onboard radar receiver.

3.4 Structured Tactical Lane Slots (`TacticalLaneObservationWrapper`)

The tactical wrapper transforms unstructured vehicle arrays into a driver-centric semantic feature vector of dimension 27 (or 30 with continuous control stability signals):

Index Range	Semantic Group	Feature Content & Normalization
0 - 4	Ego Vehicle State (5)	- obs[0]: $\frac{v_{ego}}{30.0}$ (normalized speed) - obs[1]: $\frac{v_{target}}{30.0}$ (target setpoint) - obs[2]: $\frac{lane}{3.0}$ (current lane index [0..3]) - obs[3]: can_lane_left (1.0 if lane > 0, else 0.0) - obs[4]: can_lane_right (1.0 if lane < 3, else 0.0)
5 - 20	Lane Slots (16)	4 lanes $\times$ 4 slot features each: - obs[base + 0]: $\frac{lead_dist}{100.0}$ (1.0 if lane empty ahead) - obs[base + 1]: $\Delta \frac{v_{lead}}{20.0}$ (relative closing velocity) - obs[base + 2]: $\frac{lag_dist}{100.0}$ (1.0 if lane empty behind) - obs[base + 3]: $\Delta \frac{v_{lag}}{20.0}$ (relative approaching velocity)
21 - 26	Corridor Heuristics (6)	- obs[21]: is_blocked (lead < 55m and lead speed < 27.5 m/s) - obs[22]: left_safe (left lead & lag both > 18.0m) - obs[23]: right_safe (right lead & lag both > 18.0m) - obs[24]: left_advantage (speed gain of left lead vs current lead) - obs[25]: right_advantage (speed gain of right lead vs current lead) - obs[26]: $\frac{TTC}{10.0}$ (time-to-collision to current lead; 1.0 if safe)
27 - 29	Continuous Stability (3)	- obs[27]: $\Delta \frac{\psi}{4}$ (heading error to lane centerline) - obs[28]: $\frac{y_{lane}}{4.0} m$ (lateral lane offset) - obs[29]: dot(psi) / 1.0 "rad/s" (numerical yaw rate)

3.5 Sliding Frame-Stacking (`FrameStackTacticalWrapper`)

To supply the policy with higher-order temporal derivatives (closing rates, acceleration trends) without the numerical instability of recurrent networks, we wrap the environment in a sliding frame buffer:

```
# env_config.py:895-930
class FrameStackTacticalWrapper(gym.Wrapper):
    def __init__(self, env: gym.Env, k: int = 3) → None:
        super().__init__(env)
        self.k = k
        self.stacked_dim = env.observation_space.shape[0] * k # 27 * 3 = 81 (or 30 * 3
= 90)
        self.observation_space = gym.spaces.Box(
            low=-1.0, high=1.0, shape=(self.stacked_dim,), dtype=np.float32
        )
        self._frames = []

    def reset(self, **kwargs):
        obs, info = self.env.reset(**kwargs)
        self._frames = [obs.copy() for _ in range(self.k)]
        return self._get_stacked_obs(), info

    def step(self, action):
        obs, reward, term, trunc, info = self.env.step(action)
        self._frames.pop(0)
        self._frames.append(obs.copy())
        return self._get_stacked_obs(), reward, term, trunc, info

    def _get_stacked_obs(self) → np.ndarray:
        return np.concatenate(self._frames, axis=0).astype(np.float32)
```

4 Classical Planning Baselines & The Front-Only Blindspot

To benchmark learning-based policies against industrial automotive standards, we implemented the canonical Intelligent Driver Model (IDM) coupled with the MOBIL lane-changing model in `baseline_classical.py`.

4.1 Mathematical Formulation of IDM

IDM computes continuous longitudinal acceleration a_{IDM} via two competing terms:

$$a_{\text{IDM}}(v, s, \Delta v) = a_{\text{max}} \left[1 - \left(\frac{v}{v_0} \right)^\delta \right] - a_{\text{max}} \left(\frac{s^*(v, \Delta v)}{s_{\text{net}}} \right)^2$$

where:

- v : current ego velocity (m/s)
- v_0 : desired free-flow cruise velocity (set to 30.0 m/s = 108 km/h)
- δ : free acceleration exponent (standard $\delta = 4$)
- a_{max} : maximum comfortable acceleration (1.5m/s²)
- b_{max} : comfortable deceleration (2.0m/s²)
- $s_{\text{net}} = \max(s - l_{\text{veh}}, 0.1\text{m})$: net bumper-to-bumper distance
- $s^*(v, \Delta v)$: dynamically calculated desired safety gap:

$$s^*(v, \Delta v) = s_0 + vT + \frac{v \cdot \Delta v}{2\sqrt{a_{\text{max}}b_{\text{max}}}}$$

where $s_0 = 5.0\text{m}$ is standstill distance, $T = 1.2\text{s}$ is safety headway, and $\Delta v = v_{\text{ego}} - v_{\text{lead}}$.

4.2 Mathematical Formulation of MOBIL

The Minimizing Overall Braking Induced by Lane Changes (MOBIL) algorithm governs lateral lane selection. A candidate lane change is approved if and only if two criteria are met:

1. Safety Criterion (Follower Protection):

The deceleration imposed on the new follower in the target lane $\tilde{a}_{\text{new lag}}$ must not exceed the maximum safe threshold $b_{\text{safe}} = 3.0\text{m/s}^2$:

$$\tilde{a}_{\text{new lag}} \geq -b_{\text{safe}}$$

2. Incentive Criterion (Acceleration Gain):

The ego vehicle's acceleration improvement, weighted by a politeness factor $p \in [0, 1]$ on surrounding followers, must exceed the switching hysteresis threshold $\Delta a_{\text{th}} = 0.2\text{m/s}^2$:

$$(a'_{\text{ego}} - a_{\text{ego}}) + p \cdot [(\tilde{a}_{\text{new lag}} - a_{\text{new lag}}) + (\tilde{a}_{\text{old lag}} - a_{\text{old lag}})] > \Delta a_{\text{th}}$$

4.3 The Front-Only Radar Catastrophic Blindspot

When evaluating IDM+MOBIL under the `front_only` sensor tier, crash rate explodes from 19% to 71%. The exact mathematical and code-level root cause lies in `baseline_classical.py:254-268`:

```
# baseline_classical.py:254-268
if lag_left is not None:
    lag_v = lag_left["v"]
    lag_gap = abs(lag_left["dx"])
    a_new_lag_pred = compute_idm_acceleration(...)
    safe_lag = a_new_lag_pred ≥ -b_safe
else:
    a_new_lag_pred = 0.0
    safe_lag = True # <--- VACUOUS TRUTH DEFENSE POINT!
```

The Vacuous Truth Blindspot Proof

Why does MOBIL crash 71% in front_only?

Under forward-only radar, all traffic behind the ego ($x_{rel} < 0$) is zeroed out by `filter_front_only()`. Consequently, `neighbors["lag_left"]` is `None`. When `lag_left` is `None`, the code executes the branch:

`safe_lag = True`

The safety criterion is vacuously satisfied! MOBIL presumes the left lane is completely empty behind it, detects an open gap ahead, and executes an aggressive lane change directly into the path of a fast-approaching vehicle traveling at 100 km/h, resulting in an unpreventable rear-end collision!

5 Neural Agent Architectures & Permutation Invariance

5.1 The Permutation Problem in Traffic Scenes

In standard Highway-Env, the observation is an ordered matrix of size $(N \times F)$, where row 0 is the ego vehicle and rows $1 \dots N - 1$ represent the $N - 1$ closest traffic vehicles.

However, the indexing of traffic vehicles in the simulator is an arbitrary artifact of list traversal:

- If Vehicle A (at 30m) and Vehicle B (at 45m) swap their internal indices in the simulator array, a flattened MLP processes them through completely different weight columns!
- An MLP must learn $N!$ factorial permutations of the same physical traffic scene, wasting representational capacity and resulting in policy brittleness.

5.2 DeepSet v2 Mathematical Formulation

Based on the Deep Sets theorem (Zaheer et al., 2017), any permutation-invariant function $f(X)$ over a set $X = \{x_1, \dots, x_N\}$ can be decomposed into:

$$f(X) = \rho\left(\bigoplus_{i=1}^N \varphi(x_i)\right)$$

where φ is an element-wise neural representation, $\oplus$ is a symmetric pooling operator, and ρ is a global decoder.

In our DeepSet v2 architecture (`deepset_v2/model.py`), we extend this theorem with Ego-Conditioning and Dual Symmetric Pooling:

$$z = \rho\left(\left[\frac{1}{N_{\max}} \sum_{i=1}^N \varphi([v_i; s_{\text{ego}}]) \parallel \max_{i=1}^N \varphi([v_i; s_{\text{ego}}]) \parallel s_{\text{ego}}\right]\right)$$

DeepSet v2 Tensor Transformation Trace:

$$\begin{aligned}
&\text{Input Tensor } x \in \mathbb{R}^{B \times 74} \\
&\quad \downarrow \text{Slice} \\
&s_{\text{ego}} \in \mathbb{R}^{B \times 4}, \quad V \in \mathbb{R}^{B \times 14 \times 4}, \quad M \in \{0, 1\}^{B \times 14} \\
&\quad \downarrow \text{Expand \& Concat} \\
&[V; s_{\text{ego}}^{\text{exp}}] \in \mathbb{R}^{B \times 14 \times 8} \xrightarrow{\varphi} H \in \mathbb{R}^{B \times 14 \times 64} \\
&\quad \downarrow \text{Dual Masked Pooling} \\
&h_{\text{sum}} = \frac{1}{14} \sum (H \odot M) \in \mathbb{R}^{B \times 64}, \quad h_{\text{max}} = \max_{\text{masked}}(H) \in \mathbb{R}^{B \times 64} \\
&\quad \downarrow \text{Concat with Ego} \\
&z_{\text{pooled}} = [h_{\text{sum}}; h_{\text{max}}; s_{\text{ego}}] \in \mathbb{R}^{B \times 132} \\
&\quad \downarrow \text{Global Decoder } \rho \text{ (3 layers, 256 units, Tanh)} \\
&z \in \mathbb{R}^{B \times 256} \xrightarrow{\text{Heads}} \text{Logits} \in \mathbb{R}^{B \times 5}, \quad V(s) \in \mathbb{R}^{B \times 1}
\end{aligned}$$

5.3 Code Walkthrough: _DeepSetEncoder

```

# deepset_v2/model.py:75-112
def forward(self, ego: torch.Tensor, rows: torch.Tensor, mask: torch.Tensor) → torch.Tensor:
    B, N, _ = rows.shape

    # 1. Ego-conditioned element input: [v_i ; ego] broadcast across N
    ego_exp = ego.unsqueeze(1).expand(B, N, 4)          # (B, N, 4)
    phi_in = torch.cat([rows, ego_exp], dim=-1)          # (B, N, 8)
    h = self.phi(phi_in)                                # (B, N, 64)

    # 2. Mask non-present vehicles
    mask_f = mask.unsqueeze(-1).float()                  # (B, N, 1)
    h_masked = h * mask_f

    # 3. Sum pooling divided by N_max (14) for scale stability
    h_sum = h_masked.sum(dim=1) / 14.0                  # (B, 64)

    # 4. Max pooling with -1e9 mask for bottleneck detection
    h_for_max = h.clone()
    h_for_max[~mask] = -1e9
    h_max = h_for_max.max(dim=1).values                  # (B, 64)
    h_max = h_max * mask.any(dim=1, keepdim=True).float()

    # 5. Concatenate pooled features + raw ego
    pooled = torch.cat([h_sum, h_max, ego], dim=-1)      # (B, 132)

    # 6. Global decoder rho (3x256 MLP)
    return self.rho(pooled)                             # (B, 256)

```

5.4 Action Masking: Eliminating Off-Road Transitions

To guarantee zero probability of selecting illegal actions (e.g. LANE_LEFT from lane 0 or LANE_RIGHT from lane 3), action logits are masked before softmax:

```

# ppo.py:138-146 & deepset_v2/model.py:196-201
if action_mask is not None:
    logits = torch.where(
        action_mask,
        logits,
        torch.tensor(-1e8, dtype=logits.dtype, device=logits.device)
    )
probs = Categorical(logits=logits)

```

Since $e^{-10^8} \equiv 0.0$, the policy cannot sample off-road maneuvers, completely eliminating road boundary termination crashes.

6 Constrained Safe-RL & PPO-Lagrangian

In high-speed highway navigation, balancing cruising speed and collision avoidance is notoriously difficult. If collision penalties are statically set in the reward function, the agent either crashes recklessly or succumbs to the floor-speed local minimum.

6.1 Constrained Markov Decision Process (CMDP)

We formalize safety as a Constrained Markov Decision Process:

$$\max_{\theta} J_{R(\pi_{\theta})} = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \gamma^t r_t \right] \quad \text{subject to} \quad J_{C(\pi_{\theta})} = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T c_t \right] \leq \kappa$$

where $c_t = \mathbb{I}(\text{collision at step } t)$ and $\kappa = 0.05$ is the maximum tolerable collision budget (5%).

Using Lagrange duality, the constrained optimization converts to an unconstrained saddle-point problem:

$$\min_{\lambda \geq 0} \max_{\theta} \mathcal{L}(\theta, \lambda) = J_{R(\pi_{\theta})} - \lambda (J_{C(\pi_{\theta})} - \kappa)$$

6.2 PID Lagrangian Multiplier Controller

Standard gradient ascent on λ ($\lambda \leftarrow [\lambda + \alpha(J_C - \kappa)]^+$) exhibits severe limit cycles and overshoot. We implement a Proportional-Integral-Derivative (PID) controller to regulate λ :

$$\begin{aligned} e_t &= J_{C(\pi_{\theta})} - \kappa \\ I_t &= \text{clip}(I_{t-1} + e_t, I_{\min}, I_{\max}) \\ D_t &= e_t - e_{t-1} \\ \lambda_t &= \max(0.0, K_p e_t + K_i I_t + K_d D_t) \end{aligned}$$

```
# train_optimal_overtaker.py:95-134
class PIDLagrangianController:
    def __init__(self, cost_limit: float = 0.05, kp: float = 0.05, ki: float = 0.001, kd: float = 0.0):
        self.cost_limit = cost_limit
        self.kp = kp
        self.ki = ki
        self.kd = kd
        self.integral = 0.0
        self.prev_error = 0.0
        self.lagrange_mult = 0.0

    def step(self, J_C: float) → tuple[float, float, float]:
        error = float(J_C - self.cost_limit)
        self.integral = float(np.clip(self.integral + error, 0.0, 100.0))
        derivative = error - self.prev_error
        self.prev_error = error

        raw_lambda = self.kp * error + self.ki * self.integral + self.kd * derivative
        self.lagrange_mult = float(max(0.0, raw_lambda))
        return self.lagrange_mult, error, self.integral
```

6.3 Dual-Critic Network Architecture & Advantage Adjustment

OptimalAgent instantiates two independent value networks:

1. Task Critic $V_{R(s)}$: estimates expected discounted performance reward.
2. Cost Critic $V_{C(s)}$: estimates expected discounted collision probability.

During PPO optimization, policy advantage $\hat{A}_t$ is computed by subtracting the Lagrangian cost advantage:

$$\hat{A}_t = \hat{A}_{R,t} - \lambda_t \cdot \hat{A}_{C,t}$$

When the policy violates the safety constraint ($J_C > 0.05$), λ escalates, penalizing any action with positive cost advantage $\hat{A}_{C,t} > 0$ and forcing the policy into conservative lane-keeping until safety is restored.

6.4 Truncation Bootstrapping GAE

A subtle flaw in standard PPO is treating timeout truncation ($t = 500$) as an absorbing terminal state.

- Termination (is_term): Collision occurs. The vehicle is destroyed. Target value is strictly zero: $V(s_{T+1}) = 0$.
- Truncation (is_trunc): 500-step time limit reached while still driving safely. Future value must be bootstrapped from $V(s_{T+1})$.

```
# ppo.py:344-365
nextnonterminal = 1.0 - is_term # Zeroed ONLY on crash

nextvalues = torch.where(
    (is_trunc.bool() & ~is_term.bool()),
    trunc_bootstrap_values[t], # Bootstrapped V(s_final)
    values[t + 1],
)
nextnoncutoff = 1.0 - torch.clamp(is_term + is_trunc, 0.0, 1.0)

delta = rewards[t] + args.gamma * nextvalues * nextnonterminal - values[t]
advantages[t] = lastgaelam = (
    delta + args.gamma * args.gae_lambda * nextnoncutoff * lastgaelam
)
```

7 Empirical Evaluation Engine & Metric Definitions

All policies are evaluated via the standardized multi-process benchmark harness `scripts/evaluate_av_metrics.py` across 100 deterministic held-out seeds (4000–4099).

7.1 Metric Mathematical Formulations

Metric Name	Mathematical Formula	Industrial Automotive Meaning
Crash Rate (CR)	$CR = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(\text{crash}_i) \times 100\%$	Primary safety metric across 100 episodes.
Clopper-Pearson 95% CI	Exact Binomial: $[B(\frac{\alpha}{2}; k, n - k + 1), B(1 - \frac{\alpha}{2}; k + 1, n - k)]$	Non-asymptotic, rigorous confidence intervals for crash proportions.
Crashes / 1000 km	$\frac{\sum \text{crashes}}{\sum \text{Distance (km)}} \times 1000$	Standard fleet safety metric used by Waymo and Cruise.
Time-to-Collision (TTC)	$TTC = \frac{d_{\text{lead}}}{v_{\text{ego}} - v_{\text{lead}}} \quad \text{if } v_{\text{ego}} > v_{\text{lead}}$	Imminent collision risk. Steps with $TTC < 2.0s$ flagged as critical.
RMS Acceleration	$RMS_a = \sqrt{\frac{1}{M} \sum_{t=1}^M \left(\frac{v_t - v_{t-1}}{\Delta t} \right)^2}$	Comfort metric measuring longitudinal driving smoothness.
RMS Jerk	$RMS_j = \sqrt{\frac{1}{M} \sum_{t=2}^M \left(\frac{a_t - a_{t-1}}{\Delta t} \right)^2}$	Second derivative of speed. High jerk causes passenger nausea.
Hard Braking Rate	Count of steps where $a_t < -3.0\text{m/s}^2$ per 100 km	Emergency deceleration maneuvers indicating poor anticipation.

7.2 Statistical Hypothesis Testing: Fisher's Exact Test

To prove that a policy's crash reduction against IDM (11%) is statistically significant and not an artifact of favorable random seeds, we compute the two-sided Fisher's Exact Test on the 2×2 contingency matrix:

Condition	Crashed	Surviving
Evaluated Policy (π)	k_π	$N_\pi - k_\pi$
IDM Only Baseline	k_{IDM}	$N_{\text{IDM}} - k_{\text{IDM}}$

The hypergeometric probability of observing this contingency distribution is:

$$p = \frac{\binom{k_\pi + k_{\text{IDM}}}{k_\pi} \binom{2N - (k_\pi + k_{\text{IDM}})}{N - k_\pi}}{\binom{2N}{N}}$$

For DeepSet v2 ($\frac{1}{100}$ vs $\frac{11}{100}$):

$$p = 0.0049 < 0.01$$

The safety superiority of DeepSet v2 is confirmed at the 99% statistical confidence level.

8 Visualization & Rendering Engine Engineering

Visualizing high-speed autonomous driving via animated GIFs is essential for qualitative review, but naive rendering pipelines suffer from severe optical illusions and headless environment crashes.

8.1 The Stroboscopic Wagon-Wheel Temporal Aliasing Bug

In Highway-Env, dashed lane markers have a default longitudinal spacing of:

$$\lambda_{\text{stripe}} = 4.33 \text{ meters}$$

At our 5 Hz policy frequency, the time delta between consecutive frames is:

$$\Delta t = 0.20 \text{ seconds}$$

Consider a vehicle cruising at $v = 21.65 \text{ m/s}$ (78.0 km/h):

$$\Delta x = v \cdot \Delta t = 21.65 \times 0.20 = 4.33 \text{ meters}$$

The Optical Illusion Explanation

Because the car advances exactly $4.33m$ per frame, each lane dash lands precisely on top of where the next dash was in the previous frame! To the human eye, the road appears completely stationary. At speeds slightly below 21.65 m/s , the dash pattern advances by less than one wavelength per frame, creating stroboscopic temporal aliasing (the wagon-wheel effect) where the road appears to move backward, falsely suggesting the car is reversing!

8.2 The Nyquist Anti-Aliasing Solution

To permanently eliminate temporal aliasing across the entire operating envelope (8 to 40 m/s), we monkey-patch Highway-Env's LaneGraphics parameters in `render_utils.py`:

```
# render_utils.py:23-40
ANTI_ALIAS_STRIPE_SPACING: float = 30.0 # 30 meters
ANTI_ALIAS_STRIPE_LENGTH: float = 15.0 # 15 meters

def apply_anti_alias_stripes() → None:
    LaneGraphics.STRIPE_SPACING = ANTI_ALIAS_STRIPE_SPACING
    LaneGraphics.STRIPE_LENGTH = ANTI_ALIAS_STRIPE_LENGTH
```

With $\lambda = 30.0m$, the Nyquist spatial threshold is $\frac{\lambda}{2} = 15.0m$. At the maximum highway speed (40.0 m/s), per-frame travel is:

$$\Delta x_{\text{max}} = 40.0 \times 0.20 = 8.0m < 15.0m$$

Because per-frame travel is strictly below the half-period Nyquist limit, optical phase aliasing is mathematically impossible.

8.3 Headless Black-Frame Workaround

When rendering in headless Docker containers or Linux servers without display servers (`render_mode='rgb_array'`), Pygame/OpenGL viewers initialize with `viewer.enabled = False`, returning an all-zero black frame on the first step. In `render_utils.py:get_rendered_frame`, we intercept the viewer object and force initialization:

```
# render_utils.py:56-65
unwrapped = env.unwrapped
if hasattr(unwrapped, "viewer") and unwrapped.viewer is not None:
    if not unwrapped.viewer.enabled:
        unwrapped.viewer.enabled = True
        frame = env.render()

assert frame.max() > 0, "Rendered frame is all-black after viewer workaround."
```

9 Line-by-Line Code Guide: Repository Map

This chapter maps every major algorithmic component to its exact source file, line number, class, and method so you can reference the codebase with precision.

9.1 env_config.py

Sensor Filtering & Observation Wrappers

- Lines 150–175 (`filter_front_only`): De-normalizes coordinates via r_x, r_y ; computes $|\text{atan2}(y, x)| \leq 60^\circ$; zeroes unobserved traffic rows.
- Lines 178–204 (`filter_short_range`): Calls `filter_front_only` and clips Euclidean distance to $\leq 35.0m$.
- Lines 206–224 (`filter_no_velocity`): Calls `filter_short_range` and zeroes columns 3 (v_x) and 4 (v_y).
- Lines 452–680 (`TacticalLaneObservationWrapper`): Generates 27-dim or 30-dim tactical slot vector. Computes `lane_leads`, `lane_lags`, `is_blocked`, `left_safe`, `right_safe`, `left_advantage`, `right_advantage`, `ttc`, and continuous features $(\Delta\psi, y_{\text{lane}}, \dot{\psi})$.
- Lines 681–704 (`_get_action_mask`): Queries `env.unwrapped.get_available_actions()` and enforces lateral action debounce lockout.
- Lines 895–930 (`FrameStackTacticalWrapper`): Maintains sliding FIFO frame buffer of size $K = 3$; concatenates into flat 81-dim or 90-dim vector.

9.2 deepset_v2/model.py and deepset_v2/env.py

Permutation-Invariant DeepSet v2

- `model.py:41–74` (`_DeepSetEncoder`): Defines φ (2-layer MLP, 64 hidden, Tanh) taking $[v_i; s_e]$ (8-dim) and ρ (3-layer MLP, 256 hidden, Tanh) taking $[h_{\text{sum}}; h_{\text{max}}; s_e]$ (132-dim).
- `model.py:75–113` (`forward`): Implements masked sum pooling divided by 14 and masked max pooling with -10^9 fill.
- `model.py:115–150` (`DeepSetAgent`): Decoupled actor and critic encoders (`_actor_enc`, `_critic_enc`) with separate linear heads (`actor_head`, `critic_head`).
- `model.py:151–172` (`_parse_obs`): Slices flat 74-dim tensor into ego $(B, 4)$, traffic features $(B, 14, 4)$, and boolean presence mask $(B, 14)$.
- `env.py:69–120` (`DeepSetObsWrapper`): Strips presence bit from ego row, normalizes coordinates, and structures 74-dim gym box observation.

9.3 train_optimal_overtaker.py

PPO-Lagrangian Constrained Optimization

- Lines 95–158 (`PIDLagrangianController`): Regulates dual multiplier λ via proportional error, clipped integral, and derivative of constraint error $J_C - 0.05$.
- Lines 200–244 (`OptimalAgent`): 3-layer Actor-Critic MLP with optional `cost_critic` (3 layers, 256 units).
- Lines 245–285 (`get_action_and_value`): Action masking with -10^8 , categorical action sampling, and dual value retrieval (V_R, V_C).
- Lines 740–775: Advantage combination: $A_{\text{net}} = A_R - \lambda \cdot A_C$, followed by PPO clipped surrogate updates.

9.4 ppo.py and ppo_lstm.py

PPO Core & Recurrent Memory

- ppo.py:77–105 (Agent): Standard 2-layer feedforward Actor-Critic with orthogonal weight initialization.
- ppo.py:338–365: GAE with Truncation Bootstrapping distinguishing `is_term` (crash) from `is_trunc` (timeout).
- ppo.py:433–470: Clipped surrogate policy loss, clipped value loss, and decoupled optimizer stepping (`opt_actor`, `opt_critic`).
- ppo_lstm.py:194–265 (RecurrentActor): Feature net + LSTM cell + actor head. Hidden state masking via $(1.0 - d) * \text{lstm_state}$ resets recurrent state on episode completion.

9.5 baseline_classical.py

Classical IDM & MOBIL Baselines

- Lines 61–155 (`extract_traffic_state`): Categorizes traffic into lead/lag neighbors for current, left, and right lanes.
- Lines 158–202 (`compute_idm_acceleration`): Evaluates IDM free acceleration and interaction braking with dynamic gap s^* .
- Lines 203–330 (`evaluate_mobil_lane_change`): Evaluates left and right candidate lane changes. Line 267 contains the `safe_lag = True` vacuous truth branch under forward-only radar.

9.6 scripts/evaluate_av_metrics.py and render_utils.py

Evaluation & Anti-Aliased Rendering

- `evaluate_av_metrics.py`:237–247: Closed-form Time-to-Collision (TTC) and critical TTC step accumulation.
- `evaluate_av_metrics.py`:316–328: Numerical acceleration and jerk derivation ($a_t = \Delta \frac{v}{0.2}$, $j_t = \Delta \frac{a}{0.2}$).
- `evaluate_av_metrics.py`:602–615: Two-sided Fisher's Exact Test against IDM and `always_SLOWER`.
- `render_utils.py`:23–40: Monkey-patching `LaneGraphics.STRIPE_SPACING = 30.0` and `STRIPE_LENGTH = 15.0`.
- `render_utils.py`:56–65: Headless container `viewer.enabled = True` workaround.

10 Oral Defense & Viva Guide: 10 Lethal Questions

Below are 10 rigorous, high-level technical questions an external examiner or senior reviewer will ask, along with the authoritative, code-backed defense.

10.0.1 Q1: "Why did you choose DeepSet over a Graph Neural Network (GNN) or Transformer?"

Defense: > "A Transformer or GNN has an $\mathcal{O}(N^2)$ computational complexity due to pairwise attention or edge message passing between all pairs of vehicles. In highway driving at 5 Hz, vehicle interaction is predominantly conditioned on relative state to the ego vehicle, not arbitrary vehicle-to-vehicle interactions. > DeepSet v2 provides strict permutation invariance with $\mathcal{O}(N)$ computational complexity using ego-conditioned element functions $\varphi([v_i; s_{\text{ego}}])$ and symmetric dual pooling ($\frac{1}{N_{\text{max}}} \sum \oplus \text{max}$). It executes in under 1 millisecond on CPU, eliminating all permutation variance while maintaining sub-millisecond inference latency."

10.0.2 Q2: "Why didn't you just use PPO with a large negative collision reward instead of PPO-Lagrangian?"

Defense: > "Static reward shaping creates an ill-conditioned multi-objective optimization problem. If the collision penalty is too lenient (e.g., -10), the agent drives aggressively and crashes frequently. If it is set too high (e.g., -50 or -100), the agent discovers the trivial 'always_SLOWER' local minimum—braking down to the 72 km/h floor speed to avoid collisions. > PPO-Lagrangian formulates a Constrained MDP where the dual multiplier λ dynamically adjusts via a PID controller based on the constraint error $J_C - \kappa$. This guarantees compliance with the 5% collision budget without sacrificing cruising speed."

10.0.3 Q3: "What is the 'kinematic floor speed' in Highway-Env and why does it invalidate naive baseline comparisons?"

Defense: > "In Highway-Env, NPC traffic travels between 23 and 27 m/s (83–97 km/h), while the ego vehicle has a minimum physical speed clamp at 20 m/s (72 km/h). An agent executing `always_SLOWER` cannot stop; it cruises at 72 km/h. Because it travels slower than all surrounding traffic, traffic drives away from it. > `always_SLOWER` achieves a 12% crash rate simply by doing nothing. Any RL benchmark claiming '12% crashes at 72 km/h' has not learned tactical overtaking—it has simply collapsed into the passive floor-speed local minimum."

10.0.4 Q4: "Explain why classical IDM+MOBIL crashes 71% under front-only radar."

Defense: > "In `baseline_classical.py`, MOBIL's lane change decision checks whether the trailing follower in the target lane will be forced to brake excessively: $\tilde{a}_{\text{new_lag}} \geq -b_{\text{safe}}$. > Under `front_only` radar, all traffic behind the ego ($x_{\text{rel}} < 0$) is filtered out by `filter_front_only()`. Consequently, `lag_left` and `lag_right` are `None`. The MOBIL implementation treats an unobserved vehicle as absent (`safe_lag = True`). The ego vehicle cuts into the adjacent lane directly in front of fast-moving trailing traffic, causing a 71% crash rate."

10.0.5 Q5: "How does action masking work in your neural network and why is it necessary?"

Defense: > "In `Agent.get_action_and_value`, we take the raw actor logits and apply `logits = torch.where(mask, logits, -1e8)`. When passed into PyTorch's `Categorical(logits=logits)`, the softmax denominator exponentiates -10^8 , which evaluates to exact floating-point zero ($e^{-10^8} = 0.0$). > This mathe-

matically prevents the policy from selecting illegal maneuvers (such as `LANE_LEFT` when already in lane 0), completely eliminating road boundary exit crashes and preventing wasted exploration outside the drivable asphalt."

10.0.6 Q6: "Why do you use separate Actor and Critic networks rather than a shared feature backbone?"

Defense: > "In PPO, sharing layers between the actor and critic introduces destructive gradient interference. Value loss gradients are typically much larger than policy surrogate loss gradients early in training; they distort the early feature representations required for policy exploration. > In both `OptimalAgent` and `DeepSetAgent`, we instantiate completely separate encoders for the actor and critic, each with their own optimizer (`opt_actor` and `opt_critic`) and orthogonal weight initialization. This eliminates gradient interference and stabilizes policy convergence."

10.0.7 Q7: "What is the difference between Termination and Truncation in GAE bootstrapping?"

Defense: > "Termination (`is_term`) occurs when the vehicle crashes. The episode ends because of physical failure, so future state value is strictly zero ($V(s_{T+1}) = 0$). > Truncation (`is_trunc`) occurs when the 500-step time limit is reached while the vehicle is still driving safely. The state is non-terminal; setting its future value to zero would falsely penalize surviving policies. In `ppo.py:345-360`, we bootstrap the value of the final observation $V(s_{\text{final}})$ and zero out the eligibility trace `lastgae_lam` so advantage estimates do not leak into the next episode."

10.0.8 Q8: "How does your frame-stacking wrapper replace an LSTM without performance loss?"

Defense: > "Recurrent architectures like LSTMs suffer from vanishing/exploding gradients, hidden-state bleeding across episode boundaries, and sequential rollout overhead. > In `FrameStackTacticalWrapper`, we stack $K = 3$ consecutive tactical observation frames into an 81-dimensional vector. Because consecutive frames are separated by $\Delta t = 0.2s$, the feedforward network directly observes first-order differences (closing velocity $\Delta \frac{x}{\Delta t}$) and second-order differences (acceleration trends $\Delta \frac{v}{\Delta t}$) through standard linear weights, achieving recurrent-level temporal reasoning with the training speed and stability of feedforward MLPs."

10.0.9 Q9: "Why was the road appearing to move backward in your visualization GIFs?"

Defense: > "Highway-Env's default lane dash spacing is 4.33 meters. At a 5 Hz rendering rate ($\Delta t = 0.2s$), a vehicle driving at $v = 21.65$ m/s (78 km/h) travels exactly 4.33m per frame. This matches the spatial period of the stripes, creating a stroboscopic optical illusion where the road appears completely stationary. > At speeds below 21.65 m/s, the pattern advances by less than one period per frame, causing the human visual cortex to interpret the shift as forward motion of the stripes, making the car appear to drive in reverse. We fixed this in `render_utils.py` by monkey-patching `LaneGraphics.STRIPE_SPACING = 30.0 "m"`, placing all operational speeds well below the Nyquist spatial limit."

10.0.10 Q10: "How do you prove that DeepSet v2's 1% crash rate is statistically superior to IDM's 11%?"

Defense: > "With $N = 100$ episodes per policy, empirical crash counts can be subject to random seed variance. We constructed a 2×2 contingency table comparing DeepSet v2 (1 crash, 99 survivals) against IDM Only (11 crashes, 89 survivals) and computed the exact two-sided Fisher's Exact Test (`scipy.stats.fisher_exact`). > The resulting p -value is $p = 0.0049$. Because $p < 0.01$, we reject the null hypothesis of equal safety at the 99% confidence level, proving that DeepSet v2's safety superiority is statistically significant."

— END OF CAPSTONE HANDBOOK & CODE DEFENSE GUIDE —